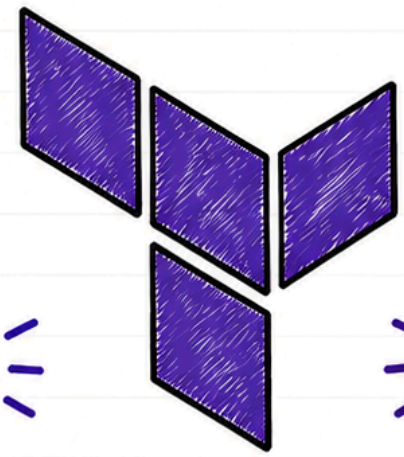
@qa_insights

TERRAFORM COMPLETE NOTES

(Terraform)

INFRASTRUCTURE AS CODE
Define, Provision & Manage Infrastructure with Code.



DECLARE & CONFIGURE
Use HCL to define your infrastructure declaratively.

PROVISION ANY INFRA
AWS, Azure, GCP, On-Prem and more.

PLAN, APPLY & MANAGE
Plan changes, review and apply with confidence.

BEGINNER TO ADVANCED

SWIPE TO NEXT →

FOLLOW | SHARE | COMMENT | LIKE

@qa_insights

1

TERRAFORM COMPLETE ROADMAP

End-to-end journey from Beginner to Advanced.

1 What is Terraform?

- Open-source IaC tool by HashiCorp.
- Define, provision & manage infrastructure as code.
- Works with multiple cloud providers & environments.

2 Why Infrastructure as Code?

- Automates infrastructure provisioning.
- Consistent, repeatable & versioned.
- Reduces manual errors.
- Speeds up delivery & reduces cost.
- Enables collaboration & scalability.

3 Where is Terraform Used?

- Cloud Infrastructure (AWS, Azure, GCP)
- Networking (VPC, Subnets, Firewall)
- Compute (VMs, Containers, Serverless)
- Databases, Storage, Load Balancers
- CI/CD, DevOps, Multi-cloud & Hybrid environments.

4 Tools You'll Learn

- Terraform CLI
- Providers (AWS, Azure, GCP, etc.)
- State Management (Local & Remote)
- Modules, Workspaces, Variables
- CI/CD Tools (GitHub Actions, Jenkins)
- Version Control (Git & GitHub)

Beginner → Advanced Roadmap

1. **Basics** — Core concepts, installation, basic commands
2. **Core Skills** — Providers, Resources, Variables, Outputs, State
3. **Intermediate** — Modules, Data Sources, Functions, Workspaces, State Management
4. **Advanced** — Remote Backend, CI/CD, Security, Best Practices
5. **Expert** — Real-world Projects, Multi-cloud, Automation, Optimization

REMEMBER

- Terraform manages infrastructure, not configuration.
- Write once, deploy anywhere.
- Code + State = Your Infrastructure.

PRO TIP

- Start small, automate everything.
- Always use version control.
- Plan → Review → Apply → Monitor.

INFRASTRUCTURE AS CODE (IaC)

Manage and provision infrastructure using code instead of manual processes.

1 What is IaC?

- Infrastructure is defined in code files.
- Version controlled & repeatable.
- Enables automation & consistency.
- Core practice of DevOps.

2 Manual vs IaC

Manual

- Clicks & console operations
- Prone to human errors
- Hard to replicate
- No version control
- Time consuming

IaC

- Code & automation
- Consistent & reliable
- Easy to replicate
- Version controlled
- Fast & scalable

3 Benefits of IaC

- Consistency across environments
- Automation & faster delivery
- Version control & collaboration
- Reduce manual errors
- Cost savings & scalability
- Easy recovery & disaster management

4 Declarative vs Imperative

Declarative

WHAT

- Define the desired state
- Tool figures out how to achieve it
- Example: Terraform, CloudFormation

Imperative

HOW

- Define step-by-step instructions
- You control the process
- Example: Bash scripts, Ansible (ad-hoc)

5 Real-world Examples

AWS, Azure, GCP Resources	Servers, VMs, Containers	Databases, Storage	Networking, Security Groups	Kubernetes Clusters	CI/CD, IAM, Load Balancers

REMEMBER

- IaC = Code + Automation + Consistency
- Treat Infra like application code
- Code once, deploy anywhere

PRO TIP

- Start small, automate repeatable tasks.
- Store code in Git.
- Use IaC for all environments.

@qa_insights

3

TERRAFORM ARCHITECTURE

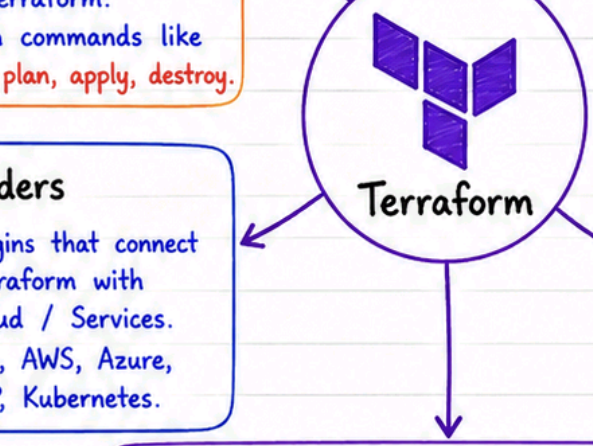
Terraform follows a simple architecture to provision & manage infrastructure.

1 Terraform CLI

- Command line tool to interact with Terraform.
- Run commands like init, plan, apply, destroy.

2 Configuration Files (.tf)

- You write code in HCL language.
- Define providers, resources, variables & outputs.



3 Providers

- Plugins that connect Terraform with Cloud / Services.
- e.g., AWS, Azure, GCP, Kubernetes.

4 Resources

- Infrastructure components.
- e.g., EC2, VPC, S3, RDS, etc.

5 State File (terraform.tfstate)

- Stores the current state of your infrastructure.
- Helps Terraform track and manage changes.

Never edit manually!

Terraform Workflow

1. **Write Code** — Define infrastructure in .tf files.
2. **Init** — Initialize working directory & providers.
3. **Plan** — Preview changes before apply.
4. **Apply** — Apply changes to create infra.
5. **Infrastructure** — Real infrastructure is created.

REMEMBER

- Code + State = Infrastructure
- State is the single source of truth
- Always review plan before apply

PRO TIP

- Store state remotely (S3, GCS, Azure Blob)
- Enable state locking
- Keep your .tf files in Version Control

TERRAFORM INSTALLATION & PROJECT STRUCTURE

Set up Terraform and organize your project the right way!

1 Install Terraform

- Download from terraform.io/downloads
- Unzip the package
- Move terraform binary to system PATH

```
# Example (macOS using Homebrew)
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

2 Verify Version

- Open terminal
- Run the command

```
terraform --version

# Example Output
Terraform v1.7.5
on darwin_arm64
```

3 Project Folder Structure

- Keep your files organized for scalability & teamwork

```
my-terraform-project/
├── .terraform/        # Auto created
├── modules/           # Reusable modules
├── environments/      # Dev, Prod, Staging
├── .gitignore         # Git ignore file
└── README.md          # Project docs
```

4 Main Files in Terraform Project

main.tf	• Defines the infrastructure resources.
variables.tf	• Declare input variables.
outputs.tf	• Define output values from resources.
providers.tf	• Configure required providers (e.g., AWS, Azure, GCP).
terraform.tfvars	• Assign values to variables. (Sensitive data can be stored here)

REMEMBER

- Terraform is a CLI tool.
- Code is written in HCL.
- Good structure = Easy to maintain.

PRO TIP

- Always use latest stable version.
- Keep code modular.
- Use .gitignore to ignore .terraform/ and *.tfstate files.

TERRAFORM CORE CONCEPTS

Building blocks that help Terraform define, provision and manage infrastructure.

1 Providers

- Plugins that connect Terraform to cloud/ platforms.
- e.g., AWS, Azure, GCP, Kubernetes.

2 Resources

- Infrastructure components.
- e.g., EC2, VPC, S3, RDS.

3 Variables

var

- Input values to make code reusable and dynamic.
- Defined by user.

4 Outputs

- Return values after resources are created.
- Useful for sharing information.

5 Data Sources

- Fetch information from existing infrastructure.
- Read-only.

6 Local Values

local

- Assign names to expressions.
- Improve readability and reuse.

7 Dependencies

- Terraform builds resources in the correct order.
- Implicit & Explicit (depends_on).

SIMPLE CODE EXAMPLE

main.tf

```
provider "aws" {
  region  = var.aws_region
}

resource "aws_s3_bucket" "example" {
  bucket  = var.bucket_name
  acl     = "private"
}

output "bucket_name" {
  value  = aws_s3_bucket.example.bucket
}
```

Provider (connects to AWS)

Resource (S3 Bucket)

Output (returns bucket name)

variables.tf

```
variable "aws_region" {
  type    = string
  default = "us-east-1"
}

variable "bucket_name" {
  type = string
}
```

REMEMBER

- Terraform reads configuration and converts it into infrastructure.
- Code = Desired State.
- State file keeps track of Real State.

PRO TIP

- Keep code modular and reusable.
- Use variables for flexibility.
- Outputs help in integration.
- Review plan before apply.

TERRAFORM WORKFLOW & COMMANDS

Write code. Plan changes. Apply with confidence.

TOP TERRAFORM COMMANDS

1. **terraform init**
   Initialize working directory and download providers.
2. **terraform validate**
   Check configuration syntax and validate files.
3. **terraform fmt**
   Format configuration files for consistency.
4. **terraform plan**
   Preview changes before applying.
5. **terraform apply**
   Apply changes and create infrastructure.
6. **terraform destroy**
   Destroy all managed infrastructure.
7. **terraform output**
   Show output values from the state.
8. **terraform show**
   Inspect the current state or a saved plan.

TERRAFORM WORKFLOW

1. **Write Code** (.tf files)
   - Define providers, resources, variables and outputs.
2. **Init** terraform init
   - Download providers and initialize the working directory.
3. **Plan** terraform plan
   - Preview the execution plan and check changes.
4. **Apply** terraform apply
   - Approve and apply changes to create infrastructure.
5. **Infrastructure Ready!**
   - Resources are provisioned and managed.

Optional Steps

- terraform output — View output values.
- terraform show — Inspect the state.

REMEMBER

- Always run init first.
- Always review plan before apply.
- State file is the source of truth.

PRO TIP

- Use version control for .tf files.
- Keep state file in remote backend.
- Automate with CI/CD for consistency.

TERRAFORM STATE MANAGEMENT

State is the memory of your infrastructure.
Manage it well, avoid chaos!

1 WHAT IS terraform.tfstate?

terraform.tfstate

- A JSON file that stores the current state of your infrastructure.
- Tracks resources, their attributes and metadata.
- Generated automatically by Terraform.

2 WHY STATE MATTERS

- Maps real infrastructure to your code.
- Enables plan, apply and destroy.
- Helps Terraform detect drift.
- Without state, Terraform can't manage existing resources.

3 LOCAL STATE

- Default: stored locally in terraform.tfstate
- Simple, easy to get started.
- Not ideal for teams.
- Risk of state loss or corruption.

4 REMOTE STATE

- Stores state in remote backend (e.g., S3, GCS).
- Enables collaboration.
- Centralized, secure and versioned.
- Recommended for teams and production.

5 STATE LOCKING

- Prevents multiple users from modifying state at the same time.
- Avoids state corruption.
- Implemented via backend (e.g., DynamoDB, GCS lock).

6 BACKEND OVERVIEW

- Backend defines where and how state is stored.
- Common backends: S3, Azure Blob, GCS, Terraform Cloud.
- Configured in backend block.

STATE FLOW (HOW IT WORKS)

7 BEST PRACTICES

- Always use remote state for teams and production.
- Enable state locking to prevent concurrent updates.
- Store state in versioned and encrypted storage.
- Restrict access to state file (least privilege).
- Backup your state regularly.
- Never edit the state file manually.

Example Backend (S3 + DynamoDB)

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state"
    key            = "project/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

REMEMBER

- State is critical. Protect it like code.
- Remote state + locking = Safe team collaboration.
- State shows what exists, not what is in your code.

PRO TIP

- Use workspaces for multiple environments.
- Review plan carefully before apply.
- Automate backups and access policies.

VARIABLES, OUTPUTS & FUNCTIONS

Make your configurations dynamic, reusable and powerful!

1 INPUT VARIABLES

- Variables allow you to parameterize your code.
- Defined in variables.tf and used in resources.
- Makes code reusable and flexible.

```
variable "aws_region" {
  description = "AWS Region"
  type        = string
}

variable "instance_type" {
  description = "EC2 Type"
  type        = string
  default     = "t3.micro"
}
```

2 VARIABLE TYPES

- Common types:

Type	Example
string	"us-east-1"
number	3, 10, 100
bool	true / false
list(string)	["a", "b", "c"]
map(string)	{ env = "dev", team = "qa" }
object(....)	{ name = "web", port = 80 }

3 DEFAULT VALUES

- Provide default values so code works even if input is not provided.
- Can be overridden using tfvars or -var.

```
variable "env" {
  type        = string
  default     = "dev"
}

variable "tags" {
  type    = map(string)
  default = {
    project    = "Terraform"
    owner      = "QA Team"
  }
}
```

4 TFVARS (VARIABLE FILES)

- Store variable values in a file.
- Keep sensitive or environment-specific values out of code.
- Use with: -var-file="file.tfvars"

```
# dev.tfvars
aws_region          = "us-east-1"
instance_type       = "t3.small"
env                 = "dev"
tags  = {
  owner             = "QA Team"
  cost_center       = "1234"
}
```

5 OUTPUT VALUES

- Outputs expose important information about your infrastructure.
- Useful for sharing between modules or after apply.

```
output "instance_id" {
  description = "EC2 Instance ID"
  value       = aws_instance.web.id
}
output "public_ip" {
  description = "Public IP Address"
  value       = aws_instance.web.public_ip
```

6 BUILT-IN FUNCTIONS

- Terraform has many built-in functions.

Function	Example
length(list)	length(["a", "b"]) # 2
upper(string)	upper("dev") # "DEV"
lower(string)	lower("DEV") # "dev"
join(delim, list)	join(",", ["a", "b"]) # "a,b"
lookup(map, key, default)	lookup(var.tags, "env", "dev")
cidrsubnet(cidr, newbits, netnum)	cidrsubnet("10.0.0.0/16", 8, 1)
format(string, ...)	format("%s-%s", "web", 1)
element(list, index)	element(["a", "b", "c"], 1) # "b"

7 EXPRESSION EXAMPLES

String Interpolation

```
name = "web-${var.env}"

# Result: web-dev
```

Conditional (Ternary)

```
instance_type = var.env == "prod"
    ? "t3.large" : "t3.small"
# Result: t3.large (if prod)
```

For Expressions (List)

```
subnet_ids = [
  for s in var.subnets : s.id
]
```

Map Lookup

```
env = lookup(var.tags, "env",
    "dev")
# If key not found, returns "dev"
```

Using Functions

```
bucket_name = lower(
    "My-BUCKET")
# Result: my-bucket
```

REMEMBER

- Use variables to avoid hardcoding values.
- Use defaults for better reusability.
- Keep sensitive values in tfvars or environment variables.
- Outputs help in integration and visibility.
- Functions and expressions make your code DRY and powerful.

PRO TIP

- Use terraform console to test expressions.
- Use validation in variables for better input control.
- Organize variables.tf, outputs.tf and tfvars for clean structure.

@qa_insights

9

ADVANCED TERRAFORM FEATURES

Take your Terraform skills to the next level!

1 MODULES

- Reusable, maintainable and shareable units of infrastructure.
- Organize code and avoid duplication.
- Can be local or from Terraform Registry.

Example (module usage)

```
module "vpc" {
  source = "../modules/vpc"
  cidr   = "10.0.0.0/16"
  azs    = ["us-east-1a","us-east-1b"]
}
```

Benefits:

- ✓ Reusability
- ✓ Consistency
- ✓ Easier Collaboration

2 COUNT

- Creates multiple instances of a resource using count meta-argument.
- Index starts from 0.

```
resource "aws_instance" "web" {
  count         = 3
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  tags = {
      Name = "web-${count.index}"
  }
}
```

Access: aws_instance.web[0], aws_instance.web[1]

3 FOR_EACH

- Creates resources for each item in a map or set.
- Each instance is identified by a key.

```
resource "aws_s3_bucket" "b" {
  for_each = toset(["dev", "qa","prod"])
  bucket   = "my_bucket-${each.key}"
  region   = "us-east-1"
  tags = {
     Environment = each.key
  }
}
```

Access: aws_s3_bucket.b["dev"]
aws_s3_bucket.b["qa"]

4 DYNAMIC BLOCKS

- Generates nested blocks dynamically.
- Useful when block structure is repeatable.

```
resource "aws_security_group" "sg" {
  name = "example-sg"
  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
       from_port   = ingress.value.from
       to_port     = ingress.value.to
       protocol    = ingress.value.proto
       cidr_blocks = ingress.value.cidr
    }
  }
}
```

Great for complex nested configurations!

5 CONDITIONAL EXPRESSIONS

? :

- Make decisions in your configurations.
- Syntax: condition ? true_value : false_value

```
variable "env" { type = string }
locals {
  instance_type = var.env == "prod" ? "t3.large" : "t3.small"
}
resource "aws_instance" "app" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = local.instance_type
}
```

Clean and powerful decision making!

6 LIFECYCLE

- Customize resource create/update/delete behavior.

```
resource "aws_db_instance" "db" {
  identifier = "mydb"
  engine     = "mysql"
  lifecycle {
    create_before_destroy = true
    prevent_destroy       = true
    ignore_changes = [
      "tags",
      "allocated_storage"
    ]
  }
}
```

create_before_destroy: New first, then destroy old.
prevent_destroy: Protects from accidental destroy.
ignore_changes: Ignores changes for given arguments.

7 DEPENDS_ON

- Create explicit dependencies between resources.
- Terraform usually handles dependencies automatically, but use when needed.

```
resource "aws_instance" "app" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  depends_on = [aws_s3_bucket.data]
}
```

Ensures the bucket is created before the instance!

8 PROVISIONERS (OVERVIEW)

>_

- Run scripts on local or remote machine as part of resource creation.
- Use with caution.
- Prefer configuration management tools (Ansible, Chef, Puppet) for complex setups.

```
resource "aws_instance" "app" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  provisioner "remote-exec" {
      inline = [
              "sudo yum update -y",
              "sudo systemctl start httpd"
    ]
  }
  connection {
    type        = "ssh"
    user        = "ec2-user"
    private_key = file("key_pem")
  } host        = self.public_ip
}
```

REMEMBER

- Use modules for reusability.
- Prefer for_each over count when keys are important.
- Keep configurations DRY and modular.
- Understand lifecycle and dependencies to avoid surprises.

PRO TIP

- Write small, focused modules.
- Use workspaces for environment separation.
- Validate with terraform plan before apply.
- Keep state safe and always use remote state.

TERRAFORM WITH AWS (REAL PROJECT)

End-to-end Infrastructure Deployment using Terraform

END-TO-END DEPLOYMENT FLOW

1 PROVIDER

- Configure AWS provider with region.

```
provider "aws" {
  region = "us-east-1"
}
```

2 VPC

- Create a VPC with CIDR block.
- Enable DNS support.

```
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
}
```

3 SUBNET

- Create a public subnet in the VPC.

```
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true
}
```

4 SECURITY GROUP

- Allow SSH (22) and HTTP (80) inbound traffic.

```
resource "aws_security_group" "sg" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

5 EC2 INSTANCE

- Launch an EC2 instance in the public subnet.
- Attach the security group.

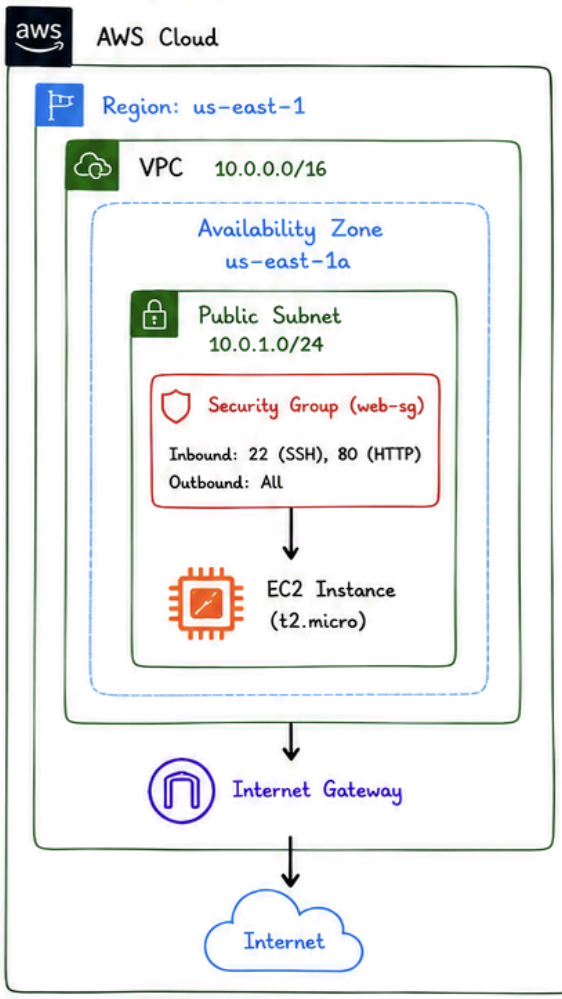
```
resource "aws_instance" "web" {
  ami                    = "ami-0c55b159cbfafe1f0"
  instance_type          = "t2.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.sg.id]
  key_name               = "my-key"
  tags = {
    Name = "Web-Server"
  }
```

6 OUTPUTS

- Show important values like public IP.

```
output "public_ip" {
  description = "Public IP of EC2"
  value       = aws_instance.web.public_ip
}
```

ARCHITECTURE DIAGRAM



PROJECT STRUCTURE

```
project/
├── main.tf              # Main resources
├── variables.tf         # Input variables
├── outputs.tf           # Output values
├── terraform.tfvars     # Variable values
└── provider.tf          # Provider configuration
```

SAMPLE VARIABLES (terraform.tfvars)

```
aws_region      = "us-east-1"
ami_id          = "ami-0c55b159cbfafe1f0"
instance_type   = "t2.micro"
key_name        = "my-key"
vpc_cidr        = "10.0.0.0/16"
public_subnet_cidr = "10.0.1.0/24"
```

SAMPLE OUTPUT (terraform output)

```
public_ip   = "3.21.45.67"
instance_id = "i-0abc1234def567890"
vpc_id      = "vpc-0a1b2c3d4e5f6789"
subnet_id   = "subnet-0a1b2c3d4e5f6789"
sg_id       = "sg-0a1b2c3d4e5f6789"
```

DEPLOYMENT COMMANDS

```
terraform init      # Initialize
terraform validate  # Validate code
terraform plan      # Preview changes
terraform apply     # Create infrastructure
terraform output    # Show outputs
terraform destroy   # Destroy resources
```

KEY TAKEAWAY

- Terraform helps you build AWS infrastructure as code.
- This setup launches a secure EC2 instance in a VPC.
- Easily reusable, version-controlled & scalable.

PRO TIP

- Use remote state (S3 + DynamoDB) for team collaboration.
- Always review plan before apply.
- Follow least privilege principle in security groups.

CI/CD & Enterprise Terraform

Automate, collaborate and scale your infrastructure with confidence!

1 GITHUB

- Store Terraform code in GitHub repositories.
- Use branches for feature development.
- Pull Requests for code review.

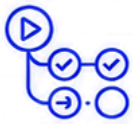
```
repo/
├── main.tf
├── variables.tf
├── outputs.tf
├── modules/
│   └── ...
└── environments/
    ├── dev/
    ├── qa/
    └── prod/
```

2 JENKINS

- Traditional CI/CD server.
- Install Terraform CLI plugin.
- Build jobs for Plan & Apply.
- Manage credentials securely.
- Publish reports & logs.

Checkout → Init → Validate → Plan → Review → Apply

3 GITHUB ACTIONS



- CI/CD directly in GitHub.
- YAML based workflows.
- Easy, fast & scalable.
- Ideal for modern teams.

```yaml
- name: Terraform Plan
  run: terraform plan -out=tfplan

- name: Terraform Apply
  run: terraform apply -auto-approve tfplan
```

4 TERRAFORM CLOUD

- Managed Terraform by HashiCorp.
- Remote operations (Plan/Apply).
- Policy & Sentinel integration,
- Run Tasks, Cost Estimation, Drift Detection.

5 REMOTE BACKEND

- Store state remotely (e.g., S3).
- Enables sharing across team.
- Provides versioning & encryption.
- Prevents state loss.

```
terraform {
  backend "s3" {
    bucket         = "my-tfstate-bucket"
    key            = "envs/prod/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "tf-locks"
    encrypt        = true
  }
}
```

6 TEAM COLLABORATION

- Multiple engineers work safely.
- Different roles & permissions.
- Shared modules & standards.
- Consistent and scalable infrastructure.

7 WORKSPACES

- Isolate environments in Terraform Cloud/Enterprise.
- Each workspace has its own state.
- Example: dev, qa, staging, prod

```
$ terraform workspace new dev
$ terraform workspace select dev
```

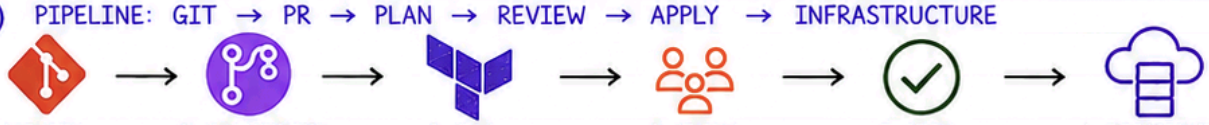
8 APPROVAL WORKFLOW

- Pull Request triggers Plan.
- Code review by team.
- Manual approval for Apply.
- Ensures quality & compliance.

9 BEST PRACTICES

- ✓ Use Remote State with locking.
- ✓ Enforce code review & approvals.
- ✓ Use workspaces for environments.
- ✓ Store secrets in Vault/Secrets Manager.
- ✓ Follow naming conventions.
- ✓ Automate Plan but control Apply.

10 PIPELINE: GIT → PR → PLAN → REVIEW → APPLY → INFRASTRUCTURE



1. **GIT PUSH** — Code pushed to GitHub repository
2. **PULL REQUEST** — Create PR for changes
3. **PLAN** — CI runs terraform plan & comments results
4. **REVIEW** — Team reviews Plan output
5. **APPLY** — Approve & apply infrastructure
6. **INFRASTRUCTURE** — Infrastructure is updated successfully!

Tools Used: GitHub / Jenkins / GitHub Actions / Terraform Cloud / S3 / DynamoDB / AWS IAM / Vault

KEY TAKEAWAY

A strong CI/CD pipeline + remote state + team collaboration = Reliable, secure & scalable Infrastructure as Code.

PRO TIP

- Automate everything you can.
- But always review before you apply!

Interview Cheat Sheet & Best Practices (Part 1)

Quick Revision for Terraform Interviews & Real-World Success!

⭐ Top Interview Topics ⭐

1 INIT vs APPLY

terraform init → terraform apply

- terraform init → Initializes the working directory. Downloads providers, sets up backend.
- terraform apply → Creates/updates infrastructure as per configuration.

2 COUNT vs FOR_EACH

```
resource "aws_instance" "web" {
  count = 3
  ...
}
```

VS

```
resource "aws_instance" "web" {
  for_each = toset(["a", "b", "c"])
  ...
}
```

- count → Uses numeric index (0,1,2...).
- for_each → Uses keys/values from a map or set.
- Use count for simple, identical resources.
- Use for_each for unique, key-based resources.

3 VARIABLES vs LOCALS

Variables

- Input to modules
- Can be customized
- Defined using variable block

VS

Locals

- Internal values
- Immutable
- Defined using locals block

Example

```
variable "region" {
  type    = string
  default = "us-east-1"
}
```

→

```
locals {
  name = "my-app"
  env  = "dev"
}
```

4 MODULES

- Reusable & shareable configuration.
- Encapsulates resources.
- Supports input variables & outputs.
- Improves maintainability & consistency.

```
module "vpc" {
  source     = "./modules/vpc"
  cidr_block = "10.0.0.0/16"
}
```

5 STATE FILE

- Stores the current state of infrastructure.
- Maps real-world resources to your configuration.
- Stored in backend (local/S3/remote).
- Critical for plan & apply operations.

```
{
  "resources": [
    { "type": "aws_instance","name": "web" }
  ]
}
```

Keep it:

- ✓ Safe
- ✓ Secured
- ✓ Backed up

BEST PRACTICES

- ✓ Use remote state with locking (S3 + DynamoDB).
- ✓ Follow naming & tagging conventions.
- ✓ Store secrets in Vault / AWS Secrets Manager.
- ✓ Use modules & version control.
- ✓ Review plan before apply.
- ✓ Keep providers & modules up to date.

PRO TIP

Golden Rule:

"Plan → Review → Apply"

Automate, Validate, Collaborate, and always keep your state safe!

Interview Cheat Sheet & Best Practices (Part 2)

Quick Revision for Terraform Interviews & Real-World Success!

6 BACKEND

- Defines where state file is stored.
- Enables team collaboration.
- Supports locking, versioning, encryption.
- Common: S3, Azure Blob, GCS, Terraform Cloud.

```
terraform {
  backend "s3" {
    bucket         = "my-tfstate-bucket"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "tf-locks"
    encrypt        = true
  }
}
```

Benefits

- ✓ Safe State
- ✓ Locking
- ✓ Versioning
- ✓ Team Access

7 WORKSPACES

- Multiple isolated environments in one state.
- Useful for dev, qa, staging, prod.
- Commands: new, list, select, delete.

```
$ terraform workspace new dev
$ terraform workspace select dev
$ terraform workspace list
```

Use Cases

- Dev/Test/Prod
- Feature branches
- Customer envs

8 IMPORT

- Imports existing infrastructure into Terraform state.
- Command: terraform import <addr> <id>
- Useful during migration.

```
$ terraform import aws_instance.web i-0abc123def456
```

Imports the existing EC2 instance into Terraform state.

9 REFRESH

- Syncs the state file with real infrastructure.
- Command: terraform refresh (auto in plan/apply)
- Detects manual changes outside Terraform.

10 DRIFT DETECTION

- Detects changes made outside Terraform.
- Command: terraform plan
- Shows what will be changed.

ADDITIONAL INTERVIEW QUICK POINTS

- ✓ init vs apply? → init sets up, apply creates/updates infra.
- ✓ count vs for_each? → count: index-based, for_each: key/value.
- ✓ Variables vs Locals? → Variables: input, Locals: internal values.
- ✓ Modules? → Reusable, shareable, promotes DRY.
- ✓ State file? → Tracks real infra, maps to config.
- ✓ Backend? → Stores state remotely, enables locking.
- ✓ Workspaces? → Manage multiple envs in same state.
- ✓ Import? → Bring existing infra under Terraform control.
- ✓ Refresh? → Sync state with real infrastructure.
- ✓ Drift Detection? → Find manual changes with plan.

KEY TAKEAWAY

- Master the core concepts.
- Follow best practices.
- Automate, collaborate & deliver with confidence!

REMEMBER

Infrastructure as Code is not just about writing code, it's about building reliable & scalable systems!